

FCFS (First Come First Serve) Scheduling – বাংলায় ব্যাখ্যা

FCFS কী? FCFS হলো সবচেয়ে সহজ CPU scheduling algorithm। এখানে process গুলোকে তাদের আসার ক্রমানুসারে execute করা হয়। অর্থাৎ যে process আগে আসবে, সেটাই আগে চলবে।

✦ কেন ব্যবহার করা হয়?

- এটি সহজ এবং বোঝা সহজ।
- ছোটখাটো সিস্টেম বা যেখানে process গুলো প্রায় সমান burst time নেয়, সেখানে এটি কার্যকর।
- কোনো জটিল calculation বা priority দরকার হয় না।

✓ উপকারিতা (Advantages)

- সহজ বাস্তবায়ন: কোড লেখা ও বোঝা সহজ।
- ন্যায্যতা (Fairness): আগে আসা process আগে চলে, তাই সবাইকে সমানভাবে দেখা হয়।
- কম overhead: অতিরিক্ত scheduling logic লাগে না।

✗ অপকারিতা (Disadvantages)

- **Convoy Effect:** যদি কোনো বড় burst time-এর process আগে আসে, তাহলে ছোট process গুলোকে অনেকক্ষণ অপেক্ষা করতে হয়।
- **Average Waiting Time বেশি হতে পারে:** বিশেষ করে যখন burst time গুলো অনেক ভিন্ন হয়।
- **Interactive system-এর জন্য ভালো নয়:** কারণ এখানে response time অনেক দেরি হতে পারে।

🌀 উদাহরণ দিয়ে বোঝানো

ধরা যাক তিনটি process আছে:

- $P1 = 10 \text{ ms}$
- $P2 = 3 \text{ ms}$
- $P3 = 2 \text{ ms}$

এখন যদি P1 আগে আসে, তবে P2 এবং P3 কে অনেকক্ষণ অপেক্ষা করতে হবে। ফলে ছোট কাজগুলো দেরিতে শেষ হবে।

👉 সংক্ষেপে, **FCFS** হলো সহজ ও ন্যায্য scheduling, কিন্তু সব পরিস্থিতিতে কার্যকর নয়। ছোট burst time-এর system বা batch processing-এর জন্য ভালো, কিন্তু interactive বা real-time system-এর জন্য উপযুক্ত নয়।

🔗 কোড ব্যাখ্যা (Line by Line)

c

```
#include <stdio.h>
```

👉 Standard input-output লাইব্রেরি ব্যবহার করা হচ্ছে, যাতে printf ও scanf কাজ করে।

c

```
int main() {
```

```
    int n, i;
```

👉 main() ফাংশন শুরু। এখানে n হলো process-এর সংখ্যা, আর i হলো loop-এর জন্য index।

c

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

👉 ইউজারকে process সংখ্যা দিতে বলা হচ্ছে।

c

```
    int bt[n], wt[n], tat[n];
```

```
    float avg_wt = 0, avg_tat = 0;
```

👉 তিনটা array তৈরি করা হলো:

- bt[] → Burst Time (প্রতিটি process কত সময় লাগবে)
- wt[] → Waiting Time
- tat[] → Turnaround Time এছাড়া average waiting time ও turnaround time হিসাব করার জন্য দুটি ভ্যারিয়েবল।

c

```
printf("Enter Burst Time for each process:\n");
```

```
for(i = 0; i < n; i++) {  
    printf("P[%d]: ", i + 1);  
    scanf("%d", &bt[i]);  
}
```

👉 প্রতিটি process-এর burst time ইউজার ইনপুট দেবে।

c

```
wt[0] = 0; // First process doesn't wait
```

👉 প্রথম process কখনো অপেক্ষা করে না, তাই তার waiting time = 0।

c

```
for(i = 1; i < n; i++) {  
    wt[i] = wt[i-1] + bt[i-1];  
}
```

👉 পরবর্তী প্রতিটি process-এর waiting time আগের process-এর waiting time + আগের burst time দিয়ে বের করা হয়।

c

```
printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
```

👉 টেবিলের হেডার প্রিন্ট করা হচ্ছে।

c

```
for(i = 0; i < n; i++) {  
    tat[i] = bt[i] + wt[i];  
    avg_wt += wt[i];  
    avg_tat += tat[i];  
    printf("P[%d]\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);  
}
```

👉 প্রতিটি process-এর turnaround time বের করা হচ্ছে:

- $tat[i] = bt[i] + wt[i]$ 🖱️ তারপর waiting time ও turnaround time যোগ করে average বের করার জন্য accumulate করা হচ্ছে। 🖱️ সবশেষে প্রতিটি process-এর তথ্য টেবিলে দেখানো হচ্ছে।

c

```
printf("\nAverage Waiting Time: %.2f", avg_wt / n);
```

```
printf("\nAverage Turnaround Time: %.2f\n", avg_tat / n);
```

🖱️ মোট waiting time ও turnaround time কে process সংখ্যা দিয়ে ভাগ করে average বের করা হচ্ছে।

c

```
return 0;
```

```
}
```

🖱️ প্রোগ্রাম শেষ।

🎯 সারসংক্ষেপ

- প্রথমে burst time ইনপুট নেওয়া হয়।
- প্রথম process-এর waiting time = 0।
- পরেরগুলো আগের burst time যোগ করে waiting time বের করে।
- Turnaround time = Burst time + Waiting time।
- শেষে average waiting time ও turnaround time বের হয়।

Round Robin Scheduling – বাংলায় ব্যাখ্যা

Round Robin কী? Round Robin হলো একটি **preemptive scheduling algorithm**। এখানে প্রতিটি process কে একটি নির্দিষ্ট সময় (Quantum বা Time Slice) দেওয়া হয়। CPU সেই সময় পর্যন্ত process চালায়, তারপর পরবর্তী process-এ যায়। যদি কোনো process শেষ না হয়, তবে সেটিকে আবার queue-তে পাঠানো হয়।

💡 কেন ব্যবহার করা হয়?

- **ন্যায্যতা (Fairness):** প্রতিটি process সমান সময় পায়।
- **Interactive system-এর জন্য ভালো:** কারণ response time কম হয়।

- **Starvation হয় না:** কোনো process অনন্তকাল অপেক্ষা করে না।

✅ উপকারিতা (Advantages)

- সব process সমান সুযোগ পায়।
- **Interactive system** (যেমন time-sharing OS) এ খুব কার্যকর।
- **Response time কমে যায়।**

❌ অপকারিতা (Disadvantages)

- **Quantum ঠিকভাবে না দিলে সমস্যা হয়:**
 - খুব ছোট হলে → অনেক context switch, overhead বেড়ে যায়।
 - খুব বড় হলে → FCFS-এর মতো হয়ে যায়।
- Average waiting time বেশি হতে পারে।

🔗 কোড ব্যাখ্যা (Line by Line)

c

```
#include <stdio.h>
```

👉 Input-output লাইব্রেরি ব্যবহার করা হচ্ছে।

c

```
int main() {
```

```
    int n, i, qt, count = 0, temp, sq = 0;
```

```
    int bt[20], wt[20], tat[20], rem_bt[20];
```

```
    float awt = 0, att = 0;
```

👉 n = process সংখ্যা, qt = Quantum Time | 👉 bt[] = Burst Time, wt[] = Waiting Time, tat[] = Turnaround Time, rem_bt[] = Remaining Burst Time | 👉 sq = মোট সময় (sum of quantum execution)।

c

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

👉 ইউজার process সংখ্যা ইনপুট দেবে।

c

```
for (i = 0; i < n; i++) {  
    printf("P[%d] Burst Time: ", i + 1);  
    scanf("%d", &bt[i]);  
    rem_bt[i] = bt[i];  
}
```

👉 প্রতিটি process-এর burst time ইনপুট নেওয়া হচ্ছে এবং একই সাথে rem_bt[] তে কপি করা হচ্ছে।

c

```
printf("Enter Quantum Time: ");  
scanf("%d", &qt);
```

👉 Quantum Time ইনপুট নেওয়া হচ্ছে।

c

```
while (1) {  
    int done = 1;  
    for (i = 0; i < n; i++) {  
        if (rem_bt[i] > 0) {  
            done = 0;  
            if (rem_bt[i] > qt) {  
                sq += qt;  
                rem_bt[i] -= qt;  
            } else {  
                sq += rem_bt[i];  
                wt[i] = sq - bt[i];  
                rem_bt[i] = 0;  
            }  
        }  
    }  
}
```

```

    }
}
}
if (done == 1) break;
}

```

👉 এখানে মূল Round Robin লজিক:

- প্রতিটি process কে quantum সময় দেওয়া হয়।
- যদি burst time quantum-এর চেয়ে বড় হয় → quantum সময় কমানো হয়, বাকি আবার queue-তে যায়।
- যদি quantum-এর মধ্যে শেষ হয়ে যায় → waiting time হিসাব করা হয়।
- done চেক করে দেখা হয় সব process শেষ হয়েছে কিনা।

```

c
printf("\nProcess\tBurst\tWaiting\tTurnaround\n");
for (i = 0; i < n; i++) {
    tat[i] = bt[i] + wt[i];
    awt += wt[i];
    att += tat[i];
    printf("P[%d]\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
}

```

👉 প্রতিটি process-এর turnaround time বের করা হচ্ছে। 👉 Average waiting time ও turnaround time হিসাব করা হচ্ছে।

```

c
printf("\nAvg Waiting Time: %.2f\nAvg Turnaround Time: %.2f\n", awt/n, att/n);
return 0;
}

```

👉 শেষে average waiting time ও turnaround time প্রিন্ট করা হচ্ছে।

সারসংক্ষেপ

- Round Robin প্রতিটি process কে সমান সময় দেয়।
- Fairness বজায় থাকে, starvation হয় না।
- Quantum ঠিকভাবে নির্বাচন করা খুব গুরুত্বপূর্ণ।

Banker's Algorithm (Deadlock Avoidance) – বাংলায় ব্যাখ্যা

Banker's Algorithm কী? Banker's Algorithm হলো একটি **Deadlock Avoidance technique**। এটি সিস্টেমকে safe state-এ রাখে। যখন কোনো process resource চাইবে, তখন algorithm দেখে নেয় যে সেই allocation করলে সিস্টেম safe থাকবে কিনা। যদি safe থাকে তবে allocate করে, নাহলে অপেক্ষা করায়।

কেন ব্যবহার করা হয়?

- Deadlock এড়ানোর জন্য।
- সিস্টেমকে সবসময় safe state-এ রাখতে।
- Resource allocation-এর সময় সঠিক সিদ্ধান্ত নিতে।

উপকারিতা (Advantages)

- Deadlock এড়ানো যায়।
- Resource allocation-এর সময় সিস্টেম crash হওয়ার সম্ভাবনা কমে।
- Safe sequence বের করা যায়।

অপকারিতা (Disadvantages)

- Algorithm জটিল এবং সময়সাপেক্ষ।
- Resource allocation-এর সময় অনেক calculation করতে হয়।
- সব সিস্টেমে ব্যবহার করা যায় না (যেমন real-time system)।

কোড ব্যাখ্যা (Line by Line)

c

```
int n, m, i, j, k;
```

```
printf("Processes: "); scanf("%d", &n);
```

```
printf("Resources: "); scanf("%d", &m);
```

👉 n = process সংখ্যা, m = resource সংখ্যা।

c

```
int alloc[n][m], max[n][m], avail[m], need[n][m], f[n], ans[n], ind = 0;
```

👉 বিভিন্ন ম্যাট্রিক্স:

- alloc[][] → প্রতিটি process কত resource পেয়েছে।
- max[][] → প্রতিটি process সর্বোচ্চ কত resource চাইতে পারে।
- avail[] → বর্তমানে কত resource available আছে।
- need[][] → প্রতিটি process-এর বাকি প্রয়োজন।
- f[] → কোন process শেষ হয়েছে কিনা চিহ্নিত করে।
- ans[] → safe sequence রাখে।

c

```
printf("Allocation Matrix:\n");
```

```
for (i = 0; i < n; i++) for (j = 0; j < m; j++) scanf("%d", &alloc[i][j]);
```

👉 Allocation matrix ইনপুট নেওয়া হচ্ছে।

c

```
printf("Max Matrix:\n");
```

```
for (i = 0; i < n; i++) for (j = 0; j < m; j++) scanf("%d", &max[i][j]);
```

👉 Max matrix ইনপুট নেওয়া হচ্ছে।

c

```
printf("Available Resources: ");
```

```
for (i = 0; i < m; i++) scanf("%d", &avail[i]);
```

👉 Available resource ইনপুট নেওয়া হচ্ছে।

c

```
for (i = 0; i < n; i++) {
```

```
f[i] = 0;
for (j = 0; j < m; j++) need[i][j] = max[i][j] - alloc[i][j];
}
```

👉 Need matrix বের করা হচ্ছে: $\text{Need} = \text{Max} - \text{Allocation}$ । 👉 সব process শুরুতে unfinished ($f[i] = 0$)।

c

```
for (k = 0; k < n; k++) {
    for (i = 0; i < n; i++) {
        if (f[i] == 0) {
            int flag = 0;
            for (j = 0; j < m; j++) {
                if (need[i][j] > avail[j]) { flag = 1; break; }
            }
            if (flag == 0) {
                ans[ind++] = i;
                for (int y = 0; y < m; y++) avail[y] += alloc[i][y];
                f[i] = 1;
            }
        }
    }
}
}
```

👉 মূল লজিক:

- প্রতিটি process চেক করা হয়।
- যদি তার $\text{need} \leq \text{available}$ হয় $\rightarrow$ allocate করা যায়।
- allocate করলে available resource বাড়ে (কারণ process শেষ হলে তার resource ফেরত আসে)।

- Safe sequence-এ সেই process যোগ হয়।

c

```
printf("\nSafe Sequence: ");
```

```
for (i = 0; i < n - 1; i++) printf(" P%d ->", ans[i]);
```

```
printf(" P%d\n", ans[n - 1]);
```

👉 Safe sequence প্রিন্ট করা হচ্ছে।

🔗 সারসংক্ষেপ

- Banker's Algorithm resource allocation simulate করে।
- Safe sequence বের করে deadlock এড়ায়।
- জটিল হলেও সিস্টেমকে নিরাপদ রাখে।

Memory Allocation (First Fit & Best Fit) – বাংলায় ব্যাখ্যা

Memory Allocation কী? যখন কোনো process মেমোরিতে জায়গা চাই, তখন Operating System নির্ধারণ করে কোন ব্লকে সেটি বসানো হবে। এজন্য বিভিন্ন algorithm ব্যবহার করা হয়। এর মধ্যে **First Fit** এবং **Best Fit** সবচেয়ে পরিচিত।

🌟 First Fit Algorithm

- Process আসার পর প্রথম যে ব্লকটি যথেষ্ট বড় পাওয়া যায়, সেটিতে process বসানো হয়।
- সহজ এবং দ্রুত কাজ করে।

উপকারিতা:

- বাস্তবায়ন সহজ।
- দ্রুত allocation হয়।

অপকারিতা:

- Fragmentation বেশি হয়।
- সবসময় optimal allocation হয় না।

🔗 কোড ব্যাখ্যা (Line by Line)

c

```
int bsize[10], psize[10], bno, pno, flags[10], allocation[10], i, j;
```

👉 bsize[] = Block size, psize[] = Process size, flags[] = ব্লক ফ্রি কিনা, allocation[] = কোন ব্লকে কোন process বসানো হয়েছে।

c

```
for(i = 0; i < 10; i++) { flags[i] = 0; allocation[i] = -1; }
```

👉 শুরুতে সব ব্লক ফ্রি (flags=0) এবং কোনো allocation হয়নি (allocation=-1)।

c

```
printf("Enter no. of blocks: "); scanf("%d", &bno);
```

```
printf("Block sizes: "); for(i = 0; i < bno; i++) scanf("%d", &bsize[i]);
```

👉 ব্লকের সংখ্যা ও সাইজ ইনপুট নেওয়া হচ্ছে।

c

```
printf("Enter no. of processes: "); scanf("%d", &pno);
```

```
printf("Process sizes: "); for(i = 0; i < pno; i++) scanf("%d", &psize[i]);
```

👉 Process সংখ্যা ও সাইজ ইনপুট নেওয়া হচ্ছে।

c

```
for(i = 0; i < pno; i++)
```

```
    for(j = 0; j < bno; j++)
```

```
        if(flags[j] == 0 && bsize[j] >= psize[i]) {
```

```
            allocation[i] = j; flags[j] = 1; break;
```

```
        }
```

👉 মূল লজিক:

- প্রতিটি process-এর জন্য প্রথম ফ্রি ব্লক খোঁজা হয়।
- যদি ব্লক যথেষ্ট বড় হয় → সেই ব্লকে বসানো হয়।
- ব্লকটি এখন ব্যস্ত (flags=1)।

c

```
printf("\nProcess No\tProcess Size\tBlock no.\n");
```

```
for(i = 0; i < pno; i++) {
    printf("%d\t\t%d\t\t", i+1, psize[i]);
    if(allocation[i] != -1) printf("%d\n", allocation[i] + 1);
    else printf("Not Allocated\n");
}
```

👉 শেষে প্রতিটি process-এর allocation দেখানো হয়। 👉 যদি কোনো ব্লক না পাওয়া যায় → "Not Allocated" প্রিন্ট হয়।

🔗 সারসংক্ষেপ

- **First Fit:** প্রথম ফ্রি ব্লক ব্যবহার করে।
- সহজ ও দ্রুত, কিন্তু fragmentation বেশি হয়।
- **Best Fit** (যেটি এখানে নেই, তবে ধারণা): সবচেয়ে ছোট ফ্রি ব্লক ব্যবহার করে যাতে জায়গা নষ্ট কম হয়।

FIFO Page Replacement – বাংলায় ব্যাখ্যা

FIFO কী? FIFO (First-In First-Out) Page Replacement হলো একটি সহজ algorithm যেখানে যখন নতুন কোনো page দরকার হয়, তখন মেমোরিতে সবচেয়ে পুরনো page (যেটি আগে ঢুকেছিল) বের করে দেওয়া হয়।

✨ কেন ব্যবহার করা হয়?

- Page fault হলে কোন page replace করতে হবে তা নির্ধারণ করার জন্য।
- সহজ এবং বাস্তবায়নযোগ্য।
- Queue-এর মতো কাজ করে: আগে আসা আগে যাবে।

✅ উপকারিতা (Advantages)

- বাস্তবায়ন সহজ।
- Fairness বজায় থাকে (সব page সমান সুযোগ পায়)।
- ছোট সিস্টেমে কার্যকর।

❌ অপকারিতা (Disadvantages)

- Optimal নয়।

- Belady's Anomaly হতে পারে (frame সংখ্যা বাড়লেও page fault বেড়ে যেতে পারে)।
- Recent ব্যবহার করা page-ও replace হয়ে যেতে পারে।

কোড ব্যাখ্যা (Line by Line)

c

```
int i, j, n, a[50], frame[10], no, k, avail, count = 0;
```

👉 n = page সংখ্যা, a[] = reference string, frame[] = memory frames, no = frame সংখ্যা, count = page fault সংখ্যা।

c

```
printf("Number of pages: "); scanf("%d", &n);
```

```
printf("Page Reference String: "); for (i = 1; i <= n; i++) scanf("%d", &a[i]);
```

```
printf("Number of frames: "); scanf("%d", &no);
```

👉 ইউজার page সংখ্যা, reference string এবং frame সংখ্যা ইনপুট দেবে।

c

```
for (i = 0; i < no; i++) frame[i] = -1;
```

```
j = 0;
```

👉 সব frame শুরুতে খালি (-1)। 👉 j হলো কোন frame replace হবে তা ট্র্যাক করার index।

c

```
printf("\nRef String\tFrames\n");
```

```
for (i = 1; i <= n; i++) {
```

```
    printf("%d\t\t", a[i]);
```

```
    avail = 0;
```

```
    for (k = 0; k < no; k++) if (frame[k] == a[i]) avail = 1;
```

👉 প্রতিটি page reference চেক করা হচ্ছে। 👉 যদি frame-এ আগে থেকেই থাকে → avail=1 (page hit)।

c

```
if (avail == 0) {
```

```

frame[j] = a[i];
j = (j + 1) % no;
count++;
for (k = 0; k < no; k++) printf("%d ", frame[k]);
}
printf("\n");

```

👉 যদি page না থাকে (page fault):

- পুরনো frame replace করা হয় ($\text{frame}[j] = a[i]$)।
- j update হয় (next frame replace হবে)।
- Page fault সংখ্যা বাড়ে।
- Frame-এর অবস্থা প্রিন্ট করা হয়।

c

```

printf("Total Page Faults: %d\n", count);
return 0;

```

👉 শেষে মোট page fault দেখানো হয়।

🎯 সারসংক্ষেপ

- FIFO Page Replacement সবচেয়ে পুরনো page replace করে।
- সহজ কিন্তু সবসময় efficient নয়।
- Belady's anomaly হতে পারে।

FCFS Disk Scheduling – বাংলায় ব্যাখ্যা

FCFS Disk Scheduling কী? এটি হলো **First Come First Serve** disk scheduling algorithm। এখানে disk head (যেটি ডাটা পড়া/লেখার জন্য ব্যবহৃত হয়) একে একে সেই request গুলো পূরণ করে যেভাবে তারা এসেছে। অর্থাৎ queue-তে যে ক্রমে request থাকে, সেই ক্রমেই head move করে।

💡 কেন ব্যবহার করা হয়?

- Disk request গুলোকে সহজভাবে handle করার জন্য।

- বাস্তুবায়ন সহজ এবং ন্যায্য (আগে আসা আগে serve হয়)।
- ছোট সিস্টেমে কার্যকর।

✅ উপকারিতা (Advantages)

- **সহজ বাস্তুবায়ন:** কোড লেখা ও বোঝা সহজ।
- **ন্যায্যতা:** আগে আসা request আগে serve হয়।
- **কম overhead:** অতিরিক্ত calculation লাগে না।

❌ অপকারিতা (Disadvantages)

- **Seek time বেশি হতে পারে:** যদি request গুলো অনেক দূরে থাকে।
- **Optimal নয়:** সবসময় কম head movement হয় না।
- **Response time বেশি হতে পারে:** বিশেষ করে যখন request গুলো random হয়।

🔍 কোড ব্যাখ্যা (Line by Line)

c

```
int queue[20], n, head, i, seek = 0, diff;
```

👉 queue[] = request list, n = request সংখ্যা, head = disk head-এর বর্তমান অবস্থান, seek = মোট head movement।

c

```
printf("Enter max range of disk: ");
```

```
int max; scanf("%d", &max);
```

👉 Disk-এর সর্বোচ্চ range ইনপুট নেওয়া হচ্ছে (যেমন 200)।

c

```
printf("Enter number of requests: "); scanf("%d", &n);
```

```
printf("Enter requests: "); for(i = 0; i < n; i++) scanf("%d", &queue[i]);
```

👉 Request সংখ্যা এবং প্রতিটি request ইনপুট নেওয়া হচ্ছে।

c

```
printf("Enter initial head position: "); scanf("%d", &head);
```

👉 Disk head-এর প্রাথমিক অবস্থান ইনপুট নেওয়া হচ্ছে।

c

```
for(i = 0; i < n; i++) {  
    diff = abs(queue[i] - head);  
    seek += diff;  
    printf("Disk head moves from %d to %d with seek %d\n", head, queue[i], diff);  
    head = queue[i];  
}
```

👉 প্রতিটি request-এর জন্য:

- Head থেকে request পর্যন্ত দূরত্ব (diff) বের করা হয়।
- সেই দূরত্ব seek-এ যোগ হয়।
- Head নতুন অবস্থানে চলে যায়।
- প্রতিটি move প্রিন্ট করা হয়।

c

```
printf("Total seek time is %d\n", seek);  
return 0;
```

👉 শেষে মোট seek time দেখানো হয়।

🎯 সারসংক্ষেপ

- FCFS Disk Scheduling request গুলোকে তাদের আসার ক্রমে serve করে।
- সহজ কিন্তু সবসময় efficient নয়।
- Seek time অনেক বেশি হতে পারে যদি request গুলো দূরে দূরে থাকে।